

通訊序列設計 期末 project

核心問題 如何找到PACP & PQCP (第二組)

范榆君

陳品維

甘承翰

Table of contents

- ODD optimal PACP
- EVEN optimal PACP
- PQCP

ODD optimal PACP

- **Condition:** For all shifts $\tau \neq 0$: $PACF_A(\tau) + PACF_B(\tau) = \pm 2$

- $\sum_{u=0}^{L-1} (\hat{\rho}(a; u) + \hat{\rho}(b; u)) = (\sum_{i=0}^{L-1} a_i)^2 + (\sum_{i=0}^{L-1} b_i)^2$

設 g_0 、 g_1 為在 A、B 序列中 1 的個數； k 為 $\sum_{u=0}^{L-1} (\hat{\rho}(a; u) + \hat{\rho}(b; u))$ 中 +2 個數

Δ 為 $\sum_{u=0}^{L-1} (\hat{\rho}(a; u) + \hat{\rho}(b; u))$ 中 (+2 個數) - (-2 個數)

左式: $\sum_{u=0}^{L-1} (\hat{\rho}(a; u) + \hat{\rho}(b; u)) = 2L + 2k - 2(L-1-k) = 4k + 2$, $0 \leq k \leq L-1 = 2L + 2\Delta$

右式: $(\sum_{i=0}^{L-1} a_i)^2 + (\sum_{i=0}^{L-1} b_i)^2 = (L - 2g_0)^2 + (L - 2g_1)^2$

可知 $2L + 2\Delta \equiv 2 \pmod{4} \Rightarrow \Delta \equiv (1-L) \pmod{4}$

使用 $(L - 2g_0)^2 + (L - 2g_1)^2 = 2L + 2\Delta$ 反推 g_0 、 g_1

Variable setting

- $\sum_{u=0}^{L-1} (\hat{\rho}(a; u) + \hat{\rho}(b; u))$ 中(+2個數)、(-2個數)以越接近 $(L-1)/2$ 為原則當基準
- EX. $L=27$

設 N_+ : +2個數、 N_- : -2個數

$$\begin{cases} N_+ + N_- = 26 \\ N_+ + N_- \equiv (1 - 27)(\text{mod } 4) \equiv 2 (\text{mod } 4) \end{cases}$$

以上面原則可解出 N_+ 、 N_- 分別為(14,12)、(12,14)

當 $(N_+, N_-) = (14, 12)$

$$\Rightarrow (L-2 \cdot g_0)^2 + (L-2 \cdot g_1)^2 = 2L + 2\Delta = 58 \Rightarrow (g_0, g_1) = (10, 12)$$

當 $(N_+, N_-) = (12, 14)$

$$\Rightarrow (L-2 \cdot g_0)^2 + (L-2 \cdot g_1)^2 = 2L + 2\Delta = 50 \Rightarrow (g_0, g_1) = (11, 11) \text{ or } (13, 10)$$

Legendre sequence

- 定義:

假設 L 是一個奇質數, 對於整數 n , Legendre Symbol $\left(\frac{n}{L}\right)$ 定義如下:

$$\left(\frac{n}{L}\right) = \begin{cases} +1, & \text{若 } n \text{ 是 } L \text{ 的二次剩餘} \\ -1, & \text{若 } n \text{ 是 } L \text{ 的非二次剩餘} \\ 0, & \text{若 } n \equiv 0 \pmod{L} \end{cases}$$

二次剩餘: 如果存在一個整數 x 使得 $x^2 \equiv n \pmod{L}$

- 序列生成:

$$a[n] = \begin{cases} \left(\frac{n}{L}\right), & \text{若 } n \not\equiv 0 \pmod{L} \\ +1, & \text{若 } n \equiv 0 \pmod{L} \text{ (或其他約定值, ex. } -1) \end{cases}$$

- 理想特性:

當 $L \equiv 3 \pmod{4}$ 時, 其 PACF 為:

$$R(\tau) = \sum_{i=0}^{L-1} a[i] * a[(i + \tau) \pmod{L}] = \begin{cases} L, & \text{若 } \tau \equiv 0 \pmod{L} \\ -1, & \text{若 } \tau \not\equiv 0 \pmod{L} \end{cases}$$

當 $L \equiv 1 \pmod{4}$ 時, 其 PACF 為:

$$R(\tau) = \sum_{i=0}^{L-1} a[i] * a[(i + \tau) \pmod{L}] = 1 \text{ or } -3$$

ex. $L=7$

$a[n]$ 其中 $n=0,1,2,3,4,5,6$

找出二次剩餘

$$1^2=1 \equiv 1 \pmod{7}$$

$$2^2=4 \equiv 4 \pmod{7}$$

$$3^2=9 \equiv 2 \pmod{7}$$

$$4^2=16 \equiv 2 \pmod{7}$$

$$5^2=25 \equiv 4 \pmod{7}$$

$$6^2=36 \equiv 1 \pmod{7}$$

$\Rightarrow \{1,2,4\} \rightarrow$ 對應值+1

$\{3,5,6\} \rightarrow$ 對應值-1

$\Rightarrow a[n] = \{+1, +1, +1, -1, +1, -1, -1\}$

Mathematical Classification

➤ **Case 1: 質數 $L \equiv 3 \pmod{4}$ (e.g., 47, 67)**

- **Legendre sequence** is **Perfect** ($\text{PACF} = -1$).
- Optimal Pair often exists with $A = B = \text{Legendre}$.
- Output: 必為 optimal odd PACP

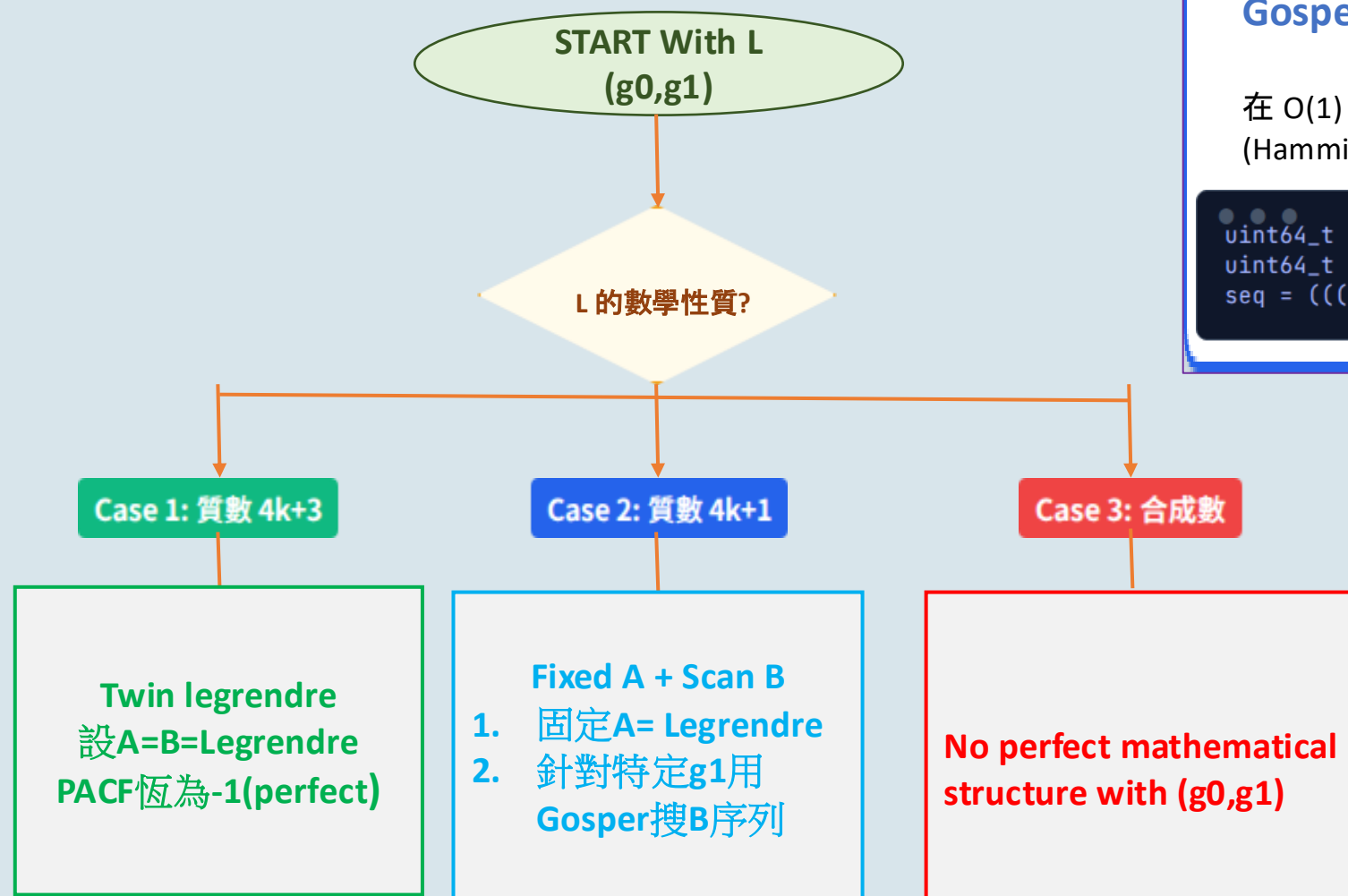
➤ **Case 2: 質數 $L \equiv 1 \pmod{4}$ (e.g., 43, 53)**

- **Legendre sequence** is **Not Flat** ($\text{PACF} = 1$ or -3).
- $A = \text{Legendre}$ works, but requires a complementary B with specific variable $g1$.

➤ **Case 3: 合數 Odd (e.g., 45, 49)**

- No perfect mathematical structure.
- Requires extensive search with specific variable ($g0, g1$).

Core algorithms



Gosper's Hack (生成器)

在 $O(1)$ 時間內生成下一個相同權重 (Hamming Weight) 的二進制排列。

```
uint64_t c = seq & -seq;  
uint64_t r = seq + c;  
seq = (((r ^ seq) >> 2) / c) | r;
```

Summary of results

Length (L)	Type	Strategy	Result Status
31 、 43 、 47 、 59 、 67 、 71...	Prime (4k+3)	Twin Legendre	Definitely Found
29 、 37 、 41 、 53	Prime (4k+1)	Legendre A + Scan B	Ex. L=53 A: {10010110111010111000000110011000000111010111011010011} B: {00000000000100011100101000110110001010100110100101111}
27 、 33 、 35 、 39 、 45 、 49 、 51 、 55 、 57	Composite	Pool A + Batch B	Ex. L=57 A: {000001100110110010100010101000110100000111011010000100011} B: {000000000000110110001111010010111010110011110010101101111}

Found: 27~59奇數 、 所有質數 $\equiv 3 \pmod{4}$ (ex. L=999 ...) for 60以上

27~59

Length (L)	(g0, g1)	Result example
27	(13, 13)	A: 11100110100101011000000000 B: 11100100110010100010100000
29	(13, 13)	A: 00001011000011001101000101011 B: 00000000000110101011001101111
31	(13, 13)	A: 0000010001110101001011110011011 B: 0010001110101001011110011011000
33	(13, 14)	A: 000011011000100101001110001010001 B: 000000000001110010011011010101111
35	(14, 15)	A: 00000010111001101000100101001000111 B: 00000000000110110100111001101010111
37	(15, 16)	A: 0000000101100110001010100111001010011 B: 0000000000010111010010111001101001111
39	(15, 19)	A: 000000110001110101001001001100000101011 B: 0000000000011010110110011001111010101111
41	(21, 16)	A: 11011001110000010101101010000011100110111 B: 000000000001010011010010100110011010001111
43	(17, 20)	A: 0000011010010010001101100010000111001010101 B: 0000000000011001100111100101101101010101111

Length (L)	(g0, g1)	Result example
45	(18, 21)	A: 000001101001000100001111010011000100100011101 B: 000000000001101011001100110100111101010101111
47	(24, 24)	A: 00001000011010100011011001001110101001111011111 B: 00001000011010100011011001001110101001111011111
49	(21, 21)	A: 0000010011101010011001011000111100100010100001101 B: 0000000000000111001100110110100101101010100011111
51	(21, 23)	A: 000001011101100101001000101010010001100001100010111 B: 000000000001010100111010011000111001101101001011111
53	(27, 21)	A: 10010110111010111000000110011000000111010111011010011 B: 00000000000011010011100101001011001001110010001010111
55	(22, 27)	A: 0000100110110000101010001011000100001100001011000111101 B: 0000000000100011110111000110111011001011101011010010111
57	(23, 28)	A: 000001100110110010100010101000110100000111011010000100011 B: 000000000000110110001111010010111010110011110010101101111
59	(30, 30)	A: 01000101011011000100001100000111110011110111001001010111011 B: 01000101011011000100001100000111110011110111001001010111011

all prime $\equiv 3(\text{mod } 4)$ (ex. L=999 ...) above 60

Length (L)	(g0, g1)	Result example
67	(34, 34)	A: 0110101100111000010100000011011101000100111111010111100011001010011 B: 0110101100111000010100000011011101000100111111010111100011001010011
71	(36, 36)	A: 0000001000101100100011100101001011100010110101100011101100101110111111 B: 0000001000101100100011100101001011100010110101100011101100101110111111
79	(40, 40)	A: 0010011000010110100000010011110011101010101000110000110111111010010111100110111 B: 0010011000010110100000010011110011101010101000110000110111111010010111100110111
83	(42, 42)	A: 010011010000111001111010100000001011000100110111001011111110101000110001111010011011 B: 010011010000111001111010100000001011000100110111001011111110101000110001111010011011
103	(52, 52)	A: 001011000111000000011101001000100010101101111011001011001000010010101110111011010001111110001110010111 B: 001011000111000000011101001000100010101101111011001011001000010010101110111011010001111110001110010111
107	(54, 54)	A: 0100111100000010110111010101001100000100001011000110011001110010111101111100110101010001001011111000011011 B: 0100111100000010110111010101001100000100001011000110011001110010111101111100110101010001001011111000011011
127	(64, 64)	A: 00101110010101000001001100111000100000110010110100101111110001011100000001011010010110011111011100011001101111101010110 0010111 B: 00101110010101000001001100111000100000110010110100101111110001011100000001011010010110011111011100011001101111101010110 0010111
131	(66, 66)	A: 11011101010111011000110001011000011110110101111011001101001111111000000001101001100100001010010000111100101110011100100 01010100010 B: 11011101010111011000110001011000011110110101111011001101001111111000000001101001100100001010010000111100101110011100100 01010100010

all prime $\equiv 3 \pmod{4}$ (ex. L=999 ...) above 60

Length (L)	(g0, g1)	Result example
139	(70, 70)	A: 110011110101010010001000110011110011111001101111010110110100000111110101000001111101001001010000100110000011000011001110 1110110101010000110 B: 110011110101010010001000110011110011111001101111010110110100000111110101000001111101001001010000100110000011000011001110 1110110101010000110
151	(76, 76)	A: 111011001111000011111110010001011010111110111101011000010011001010001100101010101100111010110011011110010100001000001010 0101110110000000111100001100100 B: 111011001111000011111110010001011010111110111101011000010011001010001100101010101100111010110011011110010100001000001010 0101110110000000111100001100100
163	(82, 82)	A: 110010100110001110000110111000000111101111010011010101111110111011000101001001000101110110110101110010001000000101010011 0100001000011111100010011110001110011010110 B: 110010100110001110000110111000000111101111010011010101111110111011000101001001000101110110110101110010001000000101010011 0100001000011111100010011110001110011010110
167	(84, 84)	A: 111110111101101010110110110111011110010100010100111100010111001111100000100111000100110111000110111110000001100010111000 01101011101011000100010010010010101001000010000 B: 111110111101101010110110110111011110010100010100111100010111001111100000100111000100110111000110111110000001100010111000 01101011101011000100010010010010101001000010000
179	(90, 90)	A: 110111000100111111011010010101010000100100110111110110001101110011111010001111001111010111000101000011000011101000001100 01001110010000010011011011110101010110100100000011011100010 B: 110111000100111111011010010101010000100100110111110110001111010001111001111010111000101000011000011101000001100 01001110010000010011011011110101010110100100000011011100010

all prime $\equiv 3(\text{mod } 4)$ (ex. L=999 ...) above 60

Length (L)	(g0, g1)	Result example
191	(96, 96)	A: 1111111011101101111010011111001010100110010110111110100001100011011100100101111100011000101000111010111001110000010110 11000100111001111010000010010110011010101011000001101000010010001000000 B: 1111111011101101111010011111001010100110010110111110100001100011011100100101111100011000101000111010111001110000010110 11000100111001111010000010010110011010101011000001101000010010001000000
199	(100, 100)	A: 1110110111100110101010010110110111011000100101110111100111100010100000011100001001111010001010111010000110111100 0111111010111000000110001100000100010110111001000100100101101010100110000100100 B: 1110110111100110101010010110110111011000100101110111100111001111100010100000011100001001111010001010111010000110111100 0111111010111000000110001100000100010110111001000100100101101010100110000100100
211	(106, 106)	A: 11001110010101101001110011000010001011000001111101011111011001011100111010010111111100100000101100111010101010001100101 1111011000000010110100011000101100100000010100000111110010111011110011000110100101011000110 B: 11001110010101101001110011000010001011000001111101011111011001011100111010010111111100100000101100111010101010001100101 1111011000000010110100011000101100100000010100000111110010111011110011000110100101011000110
223	(112, 112)	A: 11101001110000111111000001001111110111101010001011001011010101111101100111010100111001001000010001011000110011010011001 110010111011110110110001101010001100100000101010010110010111010100001000000011011111000000111000110100 B: 11101001110000111111000001001111110111101010001011001011010101111101100111010100111001001000010001011000110011010011001 110010111011110110110001101010001100100000101010010110010111010100001000000011011111000000111000110100

.

.

.

.

.

.

.

.

.

EVEN optimal PACP

Condition : Let (a, b) be a binary PACP of length L

For Even Length L :

1. Zero correliotn zone (ZCZ) is $L/2$.
2. At the time shift $u = L/2$, the magnitue of the periodic autocorrelation sum equals 4.

Meeting both of the above conditions constitutes an "Optimal" PACP.

EVEN optimal PACP

Necessary condition : For an optimal binary PACP (a, b) of even length, the following condition must hold :

$$L \pm 2 = (g_0 - g_1)^2 + (L - g_0 - g_1)^2$$

(where g_0, g_1 denote the number of ones in a and b)

For $L < 100$, no optimal binary PACP exists for the following lengths :

14, 22, 44, 46, 58, 68, 86, 90, 94

This condition can be used to quickly filter out and exclude inappropriate sequence lengths.

Permutation Pair(s) with necessary condition

Length (L)	(g0, g1)
24	(10, 9)
28	(11, 12)
30	(15, 11)
32	(15, 12)
34	(14, 14)、(17, 13)
36	(17, 14)
38	(16, 16)、(17, 15)
42	(19, 17)
48	(21, 20)、(24, 19)

濾除同構組合: ex. $L = 28$ $(g0, g1) = (11, 12)$

等效組合:

(1) 序列變號: $(11, 16)$ $(17, 12)$ $(17, 16)$

(2) A, B對調: $(16, 11)$ $(12, 17)$ $(16, 17)$ $(12, 11)$

```
int build_canonical_weight_pairs(int L, WeightPair *pairs, int max_pairs) {
    int count = 0;
    for (int g0 = 0; g0 <= L; ++g0) {
        for (int g1 = 0; g1 <= L; ++g1) {
            long long s = (long long)(g0 - g1) * (g0 - g1) + (long long)(L - g0 - g1) * (L - g0 - g1);
            if (s != (L + 2) && s != (L - 2)) continue;

            int ca = (g0 <= L - g0) ? g0 : (L - g0);
            int cb = (g1 <= L - g1) ? g1 : (L - g1);
            if (ca < cb) { int tmp = ca; ca = cb; cb = tmp; }

            if (!exists_pair(pairs, count, ca, cb)) {
                if (count < max_pairs) {
                    pairs[count].gA = ca;
                    pairs[count].gB = cb;
                    ++count;
                }
            }
        }
    }
    return count;
}
```

EVEN optimal PACP-Parameter Setup

BATCH_SIZE_A = 30000
FILTER_BOUND = 8
B_SEARCH_LIMIT = 100000000

★ FILTER BOUND設定

FILTER BOUND愈大

=> high searching range

=> high searching complexity

FILTER BOUND愈小

=> low searching range

=> filter out many PACPs

```
// =====  
#define LENGTH 48  
#define HALF_LEN 24 // L/2  
#define BATCH_SIZE_A 30000 // 增大 Batch A，最大化好 B 的利用率  
#define FILTER_BOUND 8 // 過濾界線 (建議 6-8)  
#define B_SEARCH_LIMIT 100000000LL // 給每一批 A 更多嘗試機會  
  
#define OUTPUT_FILENAME "pacp_L48_result.txt"  
  
=== PACP Search: L=48 OpenMP Parallel ===  
Pair #1: g0=24, g1=19  
Pair #2: g0=21, g1=20  
Batch Size A: 5000  
B Search Limit: All possible permutations per batch  
Filter bound: Skip A if any | PACF | > 4  
=====
```

```
===== Starting Pair #1: (g0, g1) = (24, 19) =====
```

依序搜尋不同(g0, g1) 的組合

Ex: L48 (g0,g1) 運算結果

EVEN optimal PACP - Gosper's Hack

用於搜尋空間生成 (GENERATION)

用於高效生成下一個「具有相同位元數 (SAME BIT COUNT)」的組合。

Ex. B序列中1的個數固定4

Sequence	8-bits
+Kb	01011100
-Kb	10100100
$c = Kb \& -Kb + 1$	00000100
$r = Kb + c$	01100000
$r' = r \text{ xor } Kb$	00111100
$r'' = r' \text{ shift } 2$	00001111
r'' / c	00000011
$(r'' / c) r$	01100011

← 第一組

← 第二組

Ex:

L=28

$(g0, g1) = (11, 12)$

B seq has 12 "+1"

程式碼實作

// Gosper's Hack 核心邏輯

```
unsigned __int128 c = k_b & -k_b;
```

```
unsigned __int128 r = k_b + c;
```

```
k_b = (((k_b ^ r) >> 2) / c) | r;
```

Gosper's Hack 讓我們能直接跳躍生成所有"含有12個1的28-bits整數"

EVEN optimal PACP-Algorithm



得到長度L
設定filter bound



計算(g0, g1) pair



Gosper's hack algorithm
生成固定+1, -1的
序列A



若 $|PACF_A| > \text{filter bound}(4)$
則濾除。否則放入BATCH A中

搜索全部B序列
(Gosper's hack)



先計算 $u=L/2$
PACF sum = 4



驗證 $u=1 \sim (L/2 - 1)$



EVEN Optimal PACP

VERIFY EVEN Optimal (L, Z)-SZCP

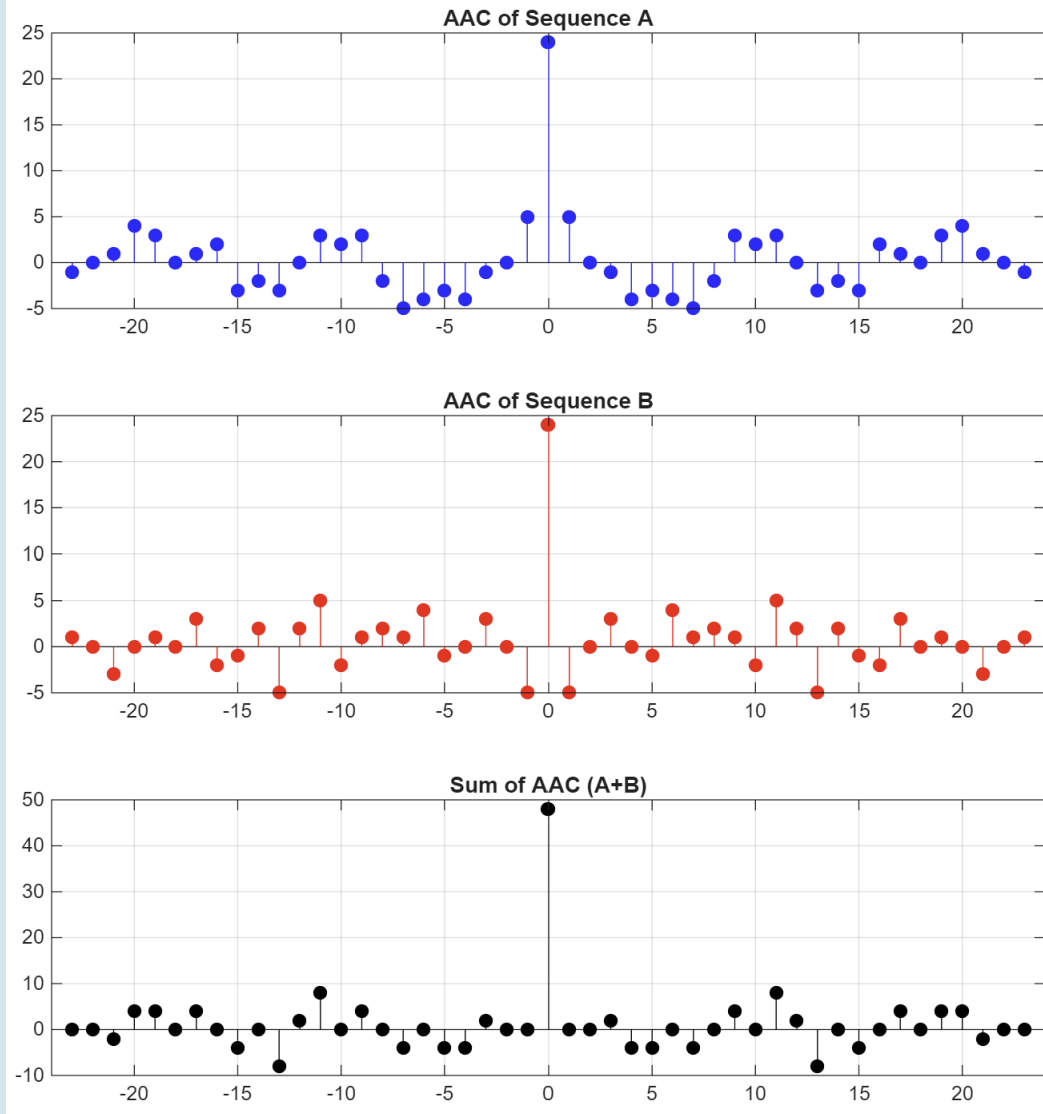
A pair of sequence ***a*** and ***b*** of even length L is called a (L, Z)-SZCP,
The optimal condition meets the following criteria:

1. maximum ZCZ width : $Z = L/2$
2. the minimum out-of-zone magnitude : 2

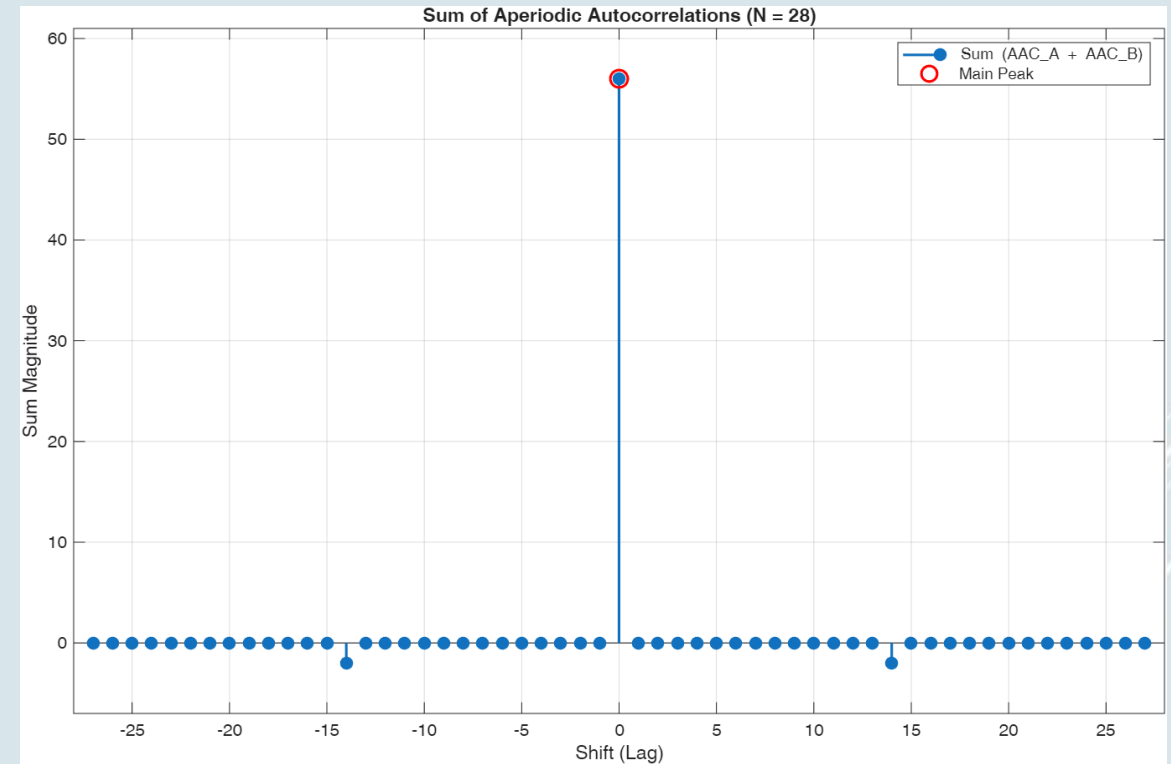
Conclusion: In cases where GCP cannot be achieved, the optimal SZCP provides the best performance boundary in sequence design.

VERIFY EVEN Optimal (L, Z)-SZCP

L = 24

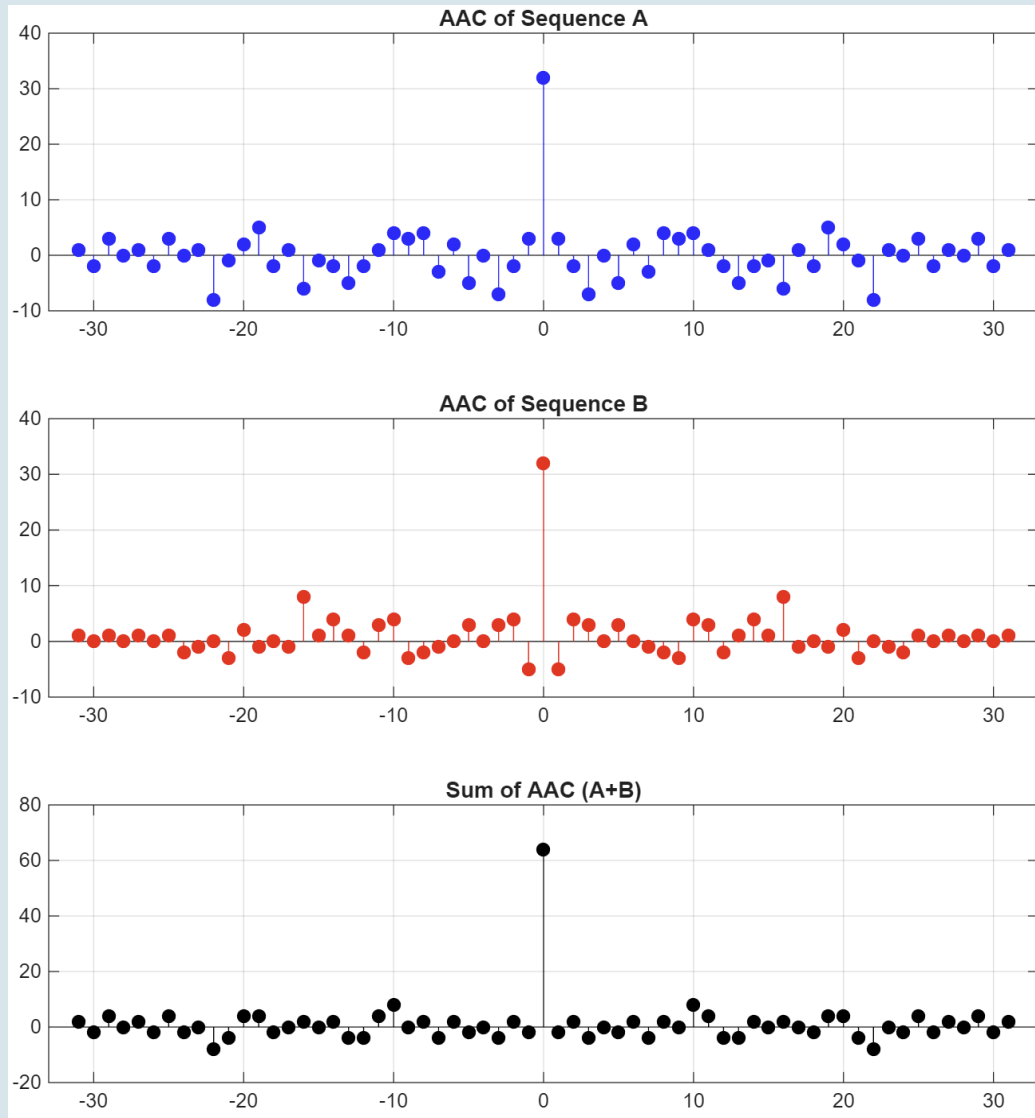


L = 28 (SZCP)

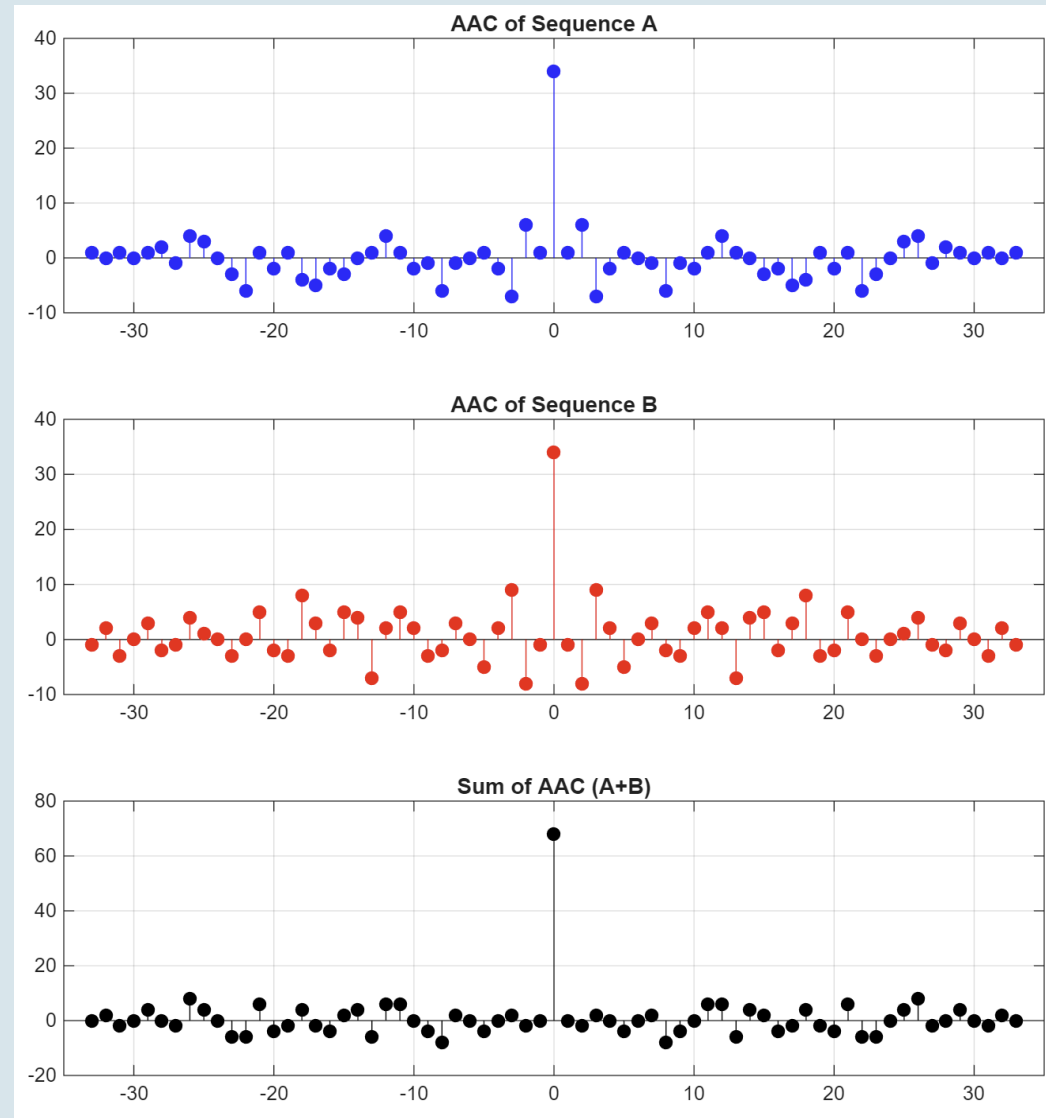


VERIFY EVEN Optimal (L, Z)-SZCP

L = 32

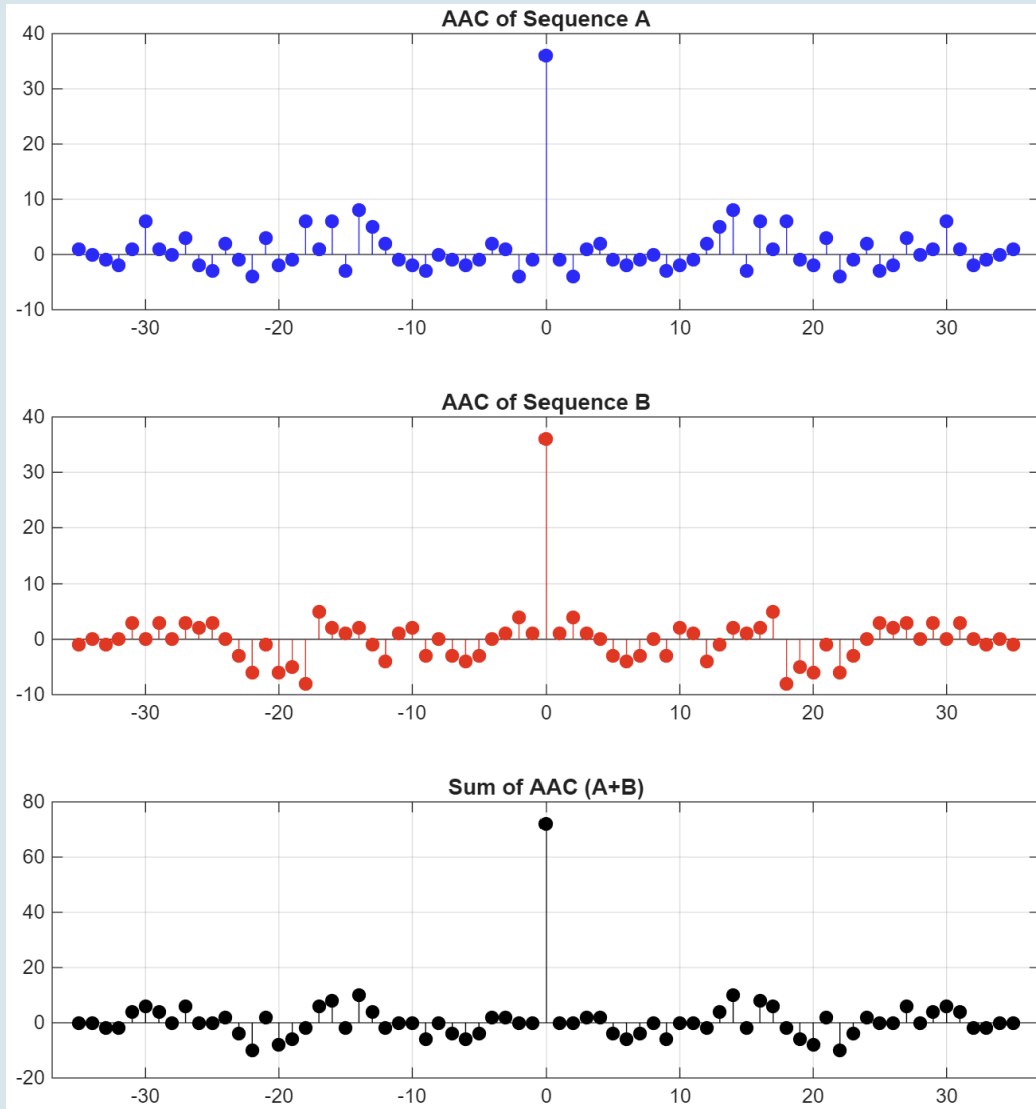


L = 34

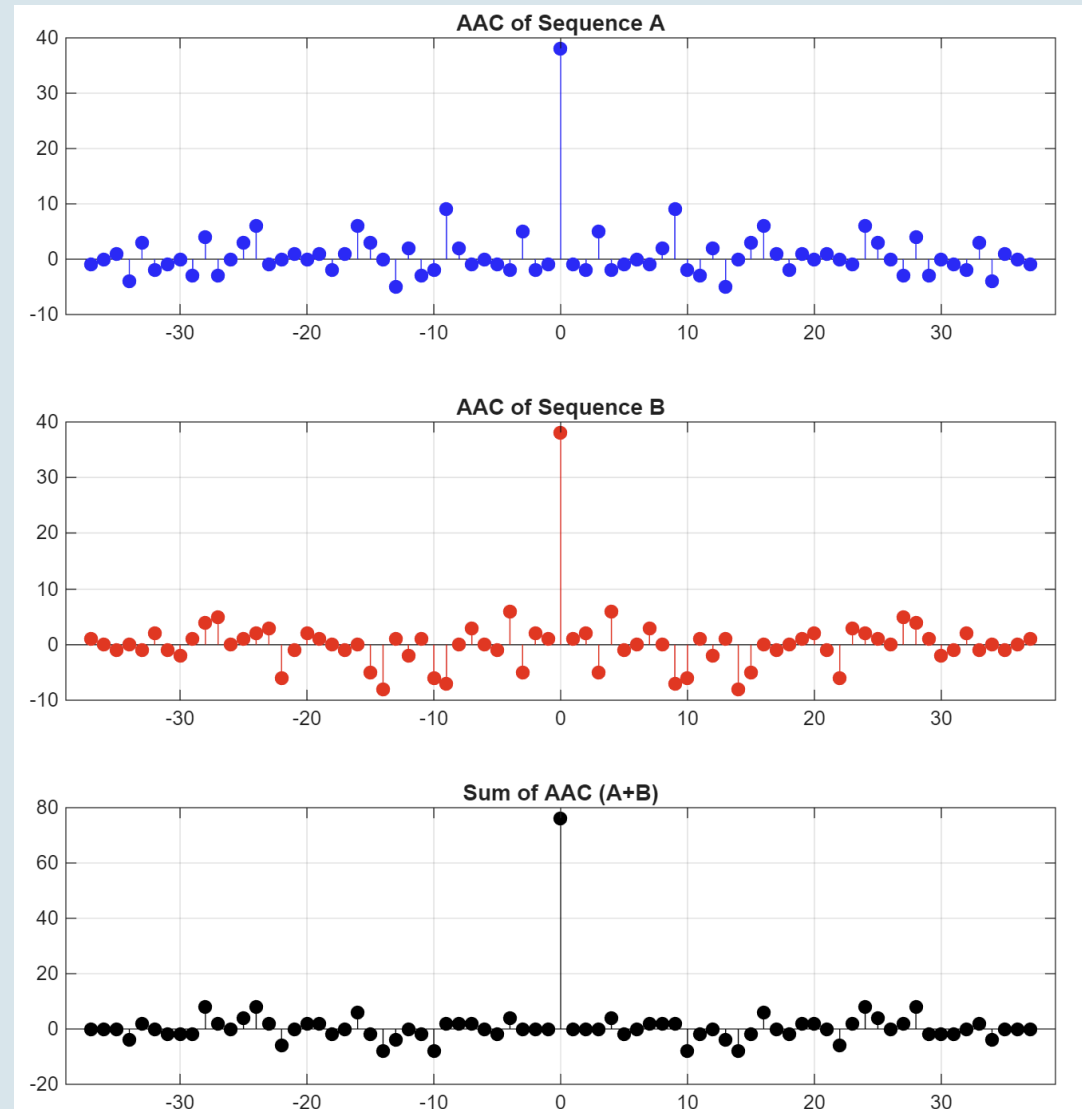


VERIFY EVEN Optimal (L, Z)-SZCP

L = 36



L = 38



Summary of results

Length (L)	(g0, g1)	Found/Not found	Result example
24	(10, 9)	Found	A: 000110010100111110000001 B: 110000100100100010101001
28	(11, 12)	Found(SZCP)	A: 0100100110001110000001101010 B: 0101011000000000111001101101
30	(15, 11)	Not found	X
32	(15, 12)	Found	A: 10111000111000000101100100011101 B: 11111110010110011101101001010101
34	(14, 14)、(17, 13)	Found	A: 0100100001111110101010111100011000 B: 1011000000011010011001001000110010
36	(17, 14)	Found	A: 100111110101100011011101110100100111 B: 101001000010111110010111011101110000
38	(16, 16)、(17, 15)	Found	A: 00101011001111011011111000011011101101 B: 10011001111100111101010100010000101111
42	(19, 17)	Not found	X
48	(21, 20)、(24, 19)	Not found	X

PQCP

Target : Find $(L,4)$ -PQCPs for $L=44,46,58,68,86,90$ and 94 .

There are two nonzero periodic autocorrelation sums, each with a magnitude of 4.

Strategy : Simulated Annealing

Simulated Annealing

1. 初始溫度(T) : 設定一個高溫, 代表允許很大的隨機性
2. 擾動(Mutation) : 隨機改變當前的序列
3. 評估(Cost) : 計算新的解好不好
4. 接受準則(Metropolis Criterion) :
 - 如果新解比較好($\Delta E < 0$) : 無條件接受
 - 如果新解比較差($\Delta E > 0$) : 看機率接受
 - 機率公式 : $P = e^{-\frac{\Delta E}{T}}$
 - 當溫度 T 很高時 : 機率 P 接近1, 容易接受爛解
 - 當溫度 T 很低時 : 機率 P 接近0, 幾乎不接受爛解
5. 降溫(Cooling) : 每一輪結束後, 把溫度將低一點

```
typedef struct { uint32_t state; } RngState;
uint32_t xorshift32(RngState *rng) {
    uint32_t x = rng->state;
    x ^= x << 13; x ^= x >> 17; x ^= x << 5;
    rng->state = x;
    return x;
}
int rand_bit(RngState *rng) { return (xorshift32(rng) & 1); }
int rand_range(RngState *rng, int range) { return xorshift32(rng) % range; }
double rand_double(RngState *rng) { return (double)xorshift32(rng) / (double)UINT32_MAX; }
```

原理 (Xorshift): 它利用位元運算的「互斥或 (XOR)」與「位移 (Shift)」來攪亂數字的位元。

- $x \ll 13$: 把數字左移13位
- $x \gg 17$: 把數字右移17位
- $x \ll 5$: 再左移5位
- (13, 17, 5): 這是一組經過數學驗證的參數, 能保證產生的數字在 $2^{32} - 1$ 次呼叫內幾乎不會重複 (Xorshift Random Number Generators)。

```

int calculate_cost(uint64_t a, uint64_t b) {
    int penalty = 0;
    int count_target = 0;

    for (int u = 1; u < L; u++) {
        uint64_t diff_a = a ^ rotate_left(a, u);
        uint64_t diff_b = b ^ rotate_left(b, u);

        int rho_a = L - 2 * (int)POPCOUNT(diff_a);
        int rho_b = L - 2 * (int)POPCOUNT(diff_b);

        int sum_mag = abs(rho_a + rho_b);

        if (sum_mag == 0) {
            continue;
        } else if (sum_mag == TARGET_MAG) {
            count_target++;
        } else {
            penalty += (sum_mag * 10);
        }
    }

    if (count_target != TARGET_COUNT) {
        penalty += abs(count_target - TARGET_COUNT) * 50;
    }
}

```

算出相關值和做互補性檢查

Penalty : 若 $|\rho_a(u) + \rho_b(u)| \neq 0 \text{ or } 4 \Rightarrow$ 累積罰分

若 非零值的數量 $\neq 2 \Rightarrow$ 累積罰分

分數越低越好，分數為 0 則代表找到解答

Result – L44

=== PQCP Search Result: SUCCESS ===

Length (L): 44

Target: Exactly 2 nonzero sums with magnitude 4

Time Elapsed: 8784.64 seconds

Restart Count: 2161, Iteration: 4322019

Sequence A: 11011100101000111101000111101101100111010011

Sequence B: 11011101010000110111010000100100101110011111

Verification Sums ($|\text{rho}(a) + \text{rho}(b)|$):

[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 0 0 0 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

Result – L46

=== PQCP Search Result: SUCCESS ===

Length (L): 46

Target: Exactly 2 nonzero sums with magnitude 4

Time Elapsed: 144794.54 seconds

Restart Count: 35988, Iteration: 4771402

Sequence A: 0001011100100101100011000110101101000101100001

Sequence B: 1000000110010011110000001101111101010100001000

Verification Sums ($|\text{rho}(a) + \text{rho}(b)|$):

[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 0 0 0 0 0 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

END